💻 Tech Assessment 02: Chatbot

AI Transformation

We hire AI Engineers at every level, intern through expert, with this one assignment. There is no separate test per level. Everyone gets this brief, and the depth and maturity of what you send back is what decides the level we offer. That is why the mandatory core is small and the depth tracks are yours to choose.

This brief is written in English. The source document is in Vietnamese, and your chatbot must answer in Vietnamese.

---

1. Context

You are building for an investor-relations analyst. They answer questions about a bank's annual report all day: from investors, from journalists, from regulators, from an executive who wants a figure before a meeting starts. Today they answer by opening the report and reading it.

Build them a chatbot that answers questions about that report.

The hard part is not the conversation. The hard part is that the analyst will paste your answer into an email to an investor. A number that is wrong, or right but attributed to the wrong page, is worse than no answer at all. It costs them credibility they cannot buy back, and they will abandon the tool the first time it happens. So every answer has to be traceable to a page the analyst can open and check in about five seconds.

Assume your user is financially literate, reads Vietnamese, and does not care what your stack is.

---

2. What to build

Six things. These are mandatory, and they are the entire required scope.

1. **Ingestion.** Turn the PDF into whatever retrievable form you think is right. The approach and the format are your decision.
2. **A chatbot.** Multi-turn, answering in Vietnamese, serving the analyst described above. Multi-turn means a follow-up like *"còn năm trước thì sao?"* has to work. A terminal REPL is perfectly acceptable; a web UI is fine but is not what we are grading.
3. **Citations.** Every factual claim in an answer carries a citation, and citations must reference the page number printed on the report page. Write each citation as `[tr. N]`, where N is that printed page number, and cite several pages as `[tr. N, M]`.

4. **Grounded refusal.** When the report does not contain the answer, the system says so instead of guessing. Some of the questions we ask are not answerable from the document.
5. **Eval numbers.** Score your system on the 10 published questions in `data/sample_questions.json`. Any method, **including scoring by hand**. Report both the number and the method. An honest manual score is worth more to us than an automatic one you never checked.
6. **Reproducibility and a decisions document.** Section 5 lists what we need in order to run your submission. `SUBMISSION_TEMPLATE.md` lists what the decisions document must cover: what you tried, what failed, what it actually costs, how fast it is, and what you would do with ten times the time and budget.

Nothing beyond these six is required. Depth is what the tracks are for.

---

3. Depth tracks: pick one or two

The core above is deliberately shallow. Pick one or two of the tracks below and go deep. Say in your submission which you picked and why. One track done well beats four done thinly, and we would rather see the second track missing than half-finished.

Document intelligence. Layout-aware chunking. Getting the tables out of the audited financial statements with their structure intact. Reading figures and charts. Deciding which of the embedded images carry information and which are decoration.

Retrieval and reasoning. Benchmarking embedding models on Vietnamese instead of defaulting to one. Hybrid lexical and dense retrieval. Reranking. Expanding abbreviations and glossary terms. Decomposing a question that needs more than one lookup. Giving the model tools to compute growth rates and ratios rather than hoping it does arithmetic correctly. Comparing figures that live in different sections.

Evaluation and observability. Building your own eval set beyond our 10. An LLM judge you validated against human judgement rather than simply trusted. Attributing a failure to retrieval versus generation. A regression harness. Tracing.

Production economics. Cost per query and per ingestion, measured rather than estimated. Caching. Routing cheap questions to cheap models. What breaks when this is 1,000 documents instead of one.

---

4. Rules

Any stack, any provider. Use your own API keys, or run entirely on local models. Both are accepted and neither is penalised. Report the cost either way. If you ran locally, report the hardware and the wall-clock time instead of a dollar figure. A solution that runs only on your own laptop is marked down. A solution that runs offline is not.

AI coding assistants are allowed and expected. We use them and we assume you do. The one condition is that you can defend every line. Shortlisted candidates sit a 45-minute walkthrough where we ask why you chose X over Y, and then ask you to change something while we watch. Code you cannot explain will be visible there within minutes, so do not ship any.

Do NOT build:

- authentication
- multi-user support
- deployment (no cloud, no orchestration, no CI)
- fine-tuning
- a design system

None of it earns credit, and an hour spent there is an hour not spent on what we grade.

---

5. How we will run your submission

We cannot assume your API keys will work for us, and we will not re-run a 197-page ingestion for every candidate. So:

- **Configure the provider through environment variables.** Document exactly how to point your system at a different key, a different provider, or a local model. Name the variables clearly and list all of them.
- **Ship the built index.** In the repository if it fits, otherwise behind a download link. Grading queries the index you shipped; we do not rebuild it.
- **One command to run it from a clean machine.** State the prerequisites, and state all of them.
- **A way to run a list of questions without a human at the keyboard.** A script, a flag, a function; your choice. We will run a larger question set against your index, and we should not have to type it in one question at a time.
- **A demo video: 3-5 minutes, unedited.** Screen recording with your voice, going through the 10 published questions. Unedited means one take. We want to see the real latency and the real failures, not a highlight reel.
- **Fill in** `SUBMISSION_TEMPLATE.md` and commit it as `SUBMISSION.md` at the root of your repository. It exists so that submissions can be compared fairly, so please keep its headings.

---

6. Things we noticed about this document

These are observations from our own reading. They are symptoms, not instructions, and the list is not exhaustive.

- **The file holds two very different kinds of content.** Narrative, marketing-shaped chapters at the front; audited financial statements with dense note tables at the back. What works well on one does not necessarily work at all on the other.
- **Vietnamese number formatting does not follow English conventions, and the units are their own problem.** The same page can carry `53,4` and `1.192` meaning very different things, and a magnitude like `nghìn tỷ đồng` has to survive intact all the way into your answer. A model applying English habits is silently wrong by a factor of a thousand, and silently is the dangerous part.
- **The file contains several hundred embedded images, and most of them carry no information.** Award logos, executive portraits, full-bleed backdrops. A few of them are not decoration.
- **The report uses abbreviations without expanding them:** `CASA`, `CIBG`, `RBG`, `N/N`, `B05/TCTD-HN`. There is a glossary near the back. An analyst asking about "CASA" and a page discussing "tiền gửi không kỳ hạn" are talking about the same thing; your retrieval may disagree.
- **The same header and footer furniture repeats throughout the report.**

---

7. Questions worth asking yourself

- What does your system do when the answer genuinely is not in the report, and how do you know it does that reliably rather than by luck?
- What should your system do when two sources disagree?
- When an answer comes out wrong, can you tell whether retrieval or generation failed? What did you have to build in order to be able to tell?
- What does one question cost, in money and in seconds? What did ingestion cost, once?
- How much of what you built is specific to this document, and how much of it survives the next hundred documents?
- Where in this pipeline should a bank not put a language model at all?
- What did you not have time to verify, and how would you have verified it?

These are our observations, not a spec. If you find a better framing, take it.

---

8. Time

Budget roughly **10-15 hours of real work, over one calendar week.** We are not measuring endurance, and we are not impressed by a weekend with no sleep in it.

A well-reasoned partial solution beats a rushed complete one. That is not a courtesy line. The rubric rewards it directly. We weight judgment, measurement and honesty at least as heavily as feature count. If you run out of time, stop, and write down what you would have done next and why. A decisions document that says *"I did not get to X; here is what I would have tried and what I expected to happen"* scores better than an X assembled in the last hour and never tested.

---

9. What you get

- `data/techcombank-bao-cao-thuong-nien-2025-vie-update.pdf`: Techcombank's 2025 annual report, in Vietnamese, 197 pages, 28 MB.
- `data/sample_questions.json`: 10 questions with gold answers, so you can calibrate.
- `SUBMISSION_TEMPLATE.md`: the skeleton to fill in and return as `SUBMISSION.md`.

The 10 published questions are a sample, not the test. Grading runs a larger held-out set against the index you shipped, drawn from the same document and covering ground the 10 do not. Tuning to the 10 shows up as a gap between your reported number and ours, and that gap is itself something we read.

Good luck. If anything in this brief is ambiguous, make a decision, write down why, and move on. That is the job.